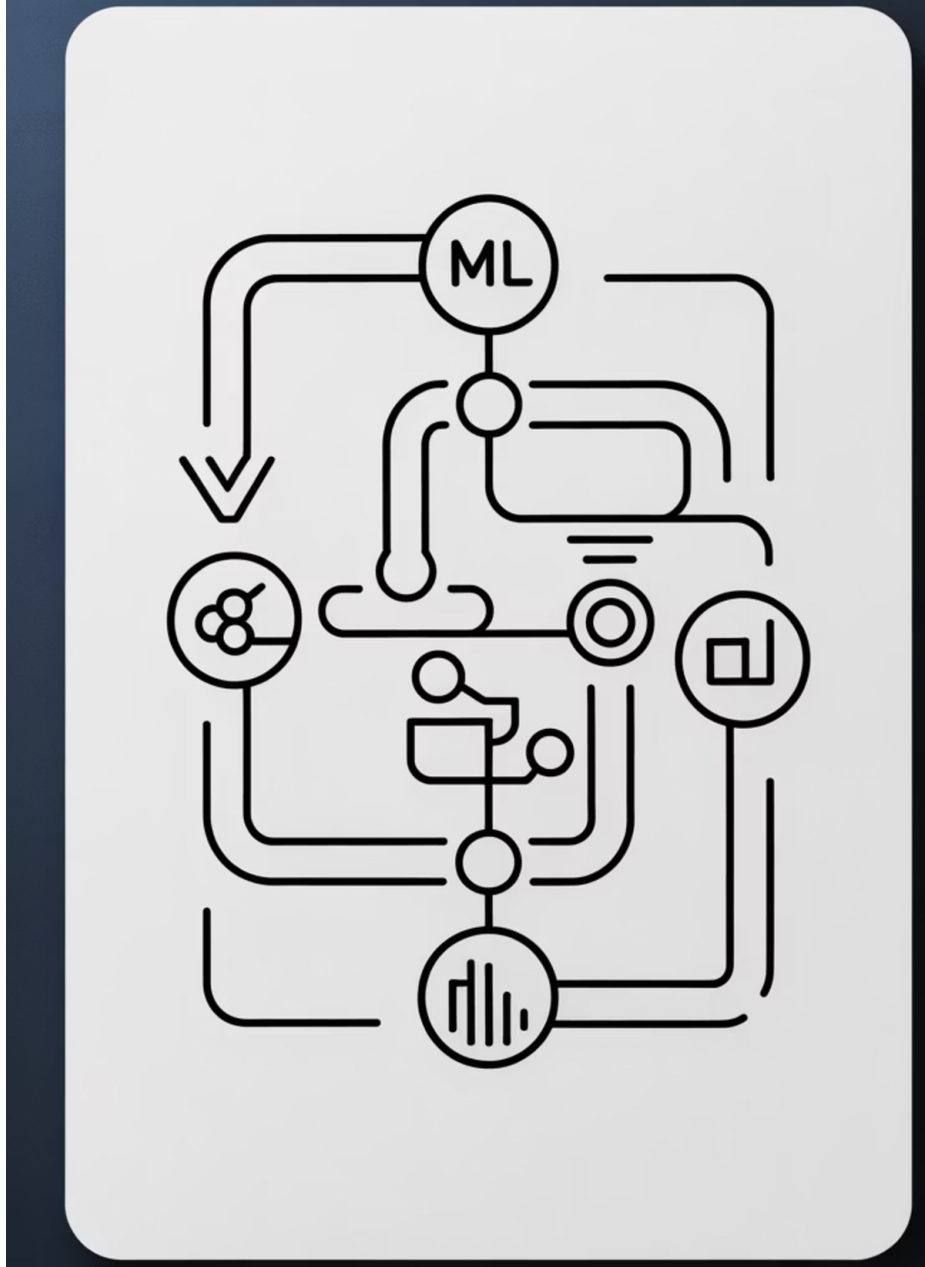
BPMN и Оркестрация жизненного цикла моделей

Навигация по жизненному циклу ML-моделей через стандарты бизнес-процессов



Зачем нам BPMN в ModelOps?

Цели:

- Научиться моделировать жизненный цикл ML-моделей как бизнес-процессы
- Использовать BPMN для оркестрации автоматизации и настройки контроля human-in-the-loop
- Интегрировать BPMN в enterprise- и open-source-стеки ModelOps

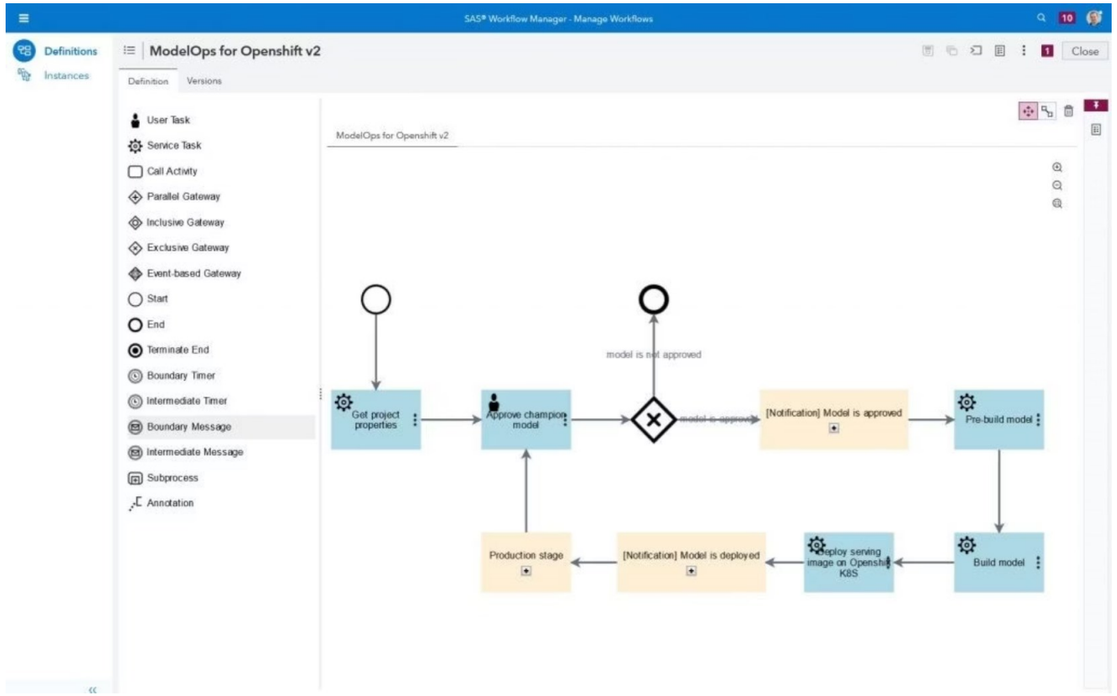
Связь с предыдущим модулем:

> В модуле «ModelOps как надмножество MLOps» мы определили ModelOps как систему управления **всем** жизненным циклом модели — от идеи до аудита. BPMN — это язык описания этих процессов, понятный и бизнесу, и инженерам.

BPMN — стандарт визуального моделирования бизнес-процессов

- **BPMN** = Business Process Model and Notation
- Стандарт от OMG (Object Management Group), версия 2.0+
- Цель: единый язык между бизнес-аналитиками, разработчиками и операционными командами
- **Основные элементы:** события, задачи, шлюзы, потоки

Почему это важно для Data Science? → Позволяет документировать, автоматизировать и мониторить процессы, в которых участвуют модели.



BPMN — Гибкая оркестрация для ModelOps

Асинхронность

Поддержка **асинхронных** и **синхронных** задач

Логика

Встроенные **условия**, **циклы**, **параллельные ветки**

Human-in-the-loop

Возможность **внедрять человека** в процесс

Интеграция

Интеграция с REST, очередями, CI/CD, уведомлениями

Аудитируемость

Каждый шаг логируется

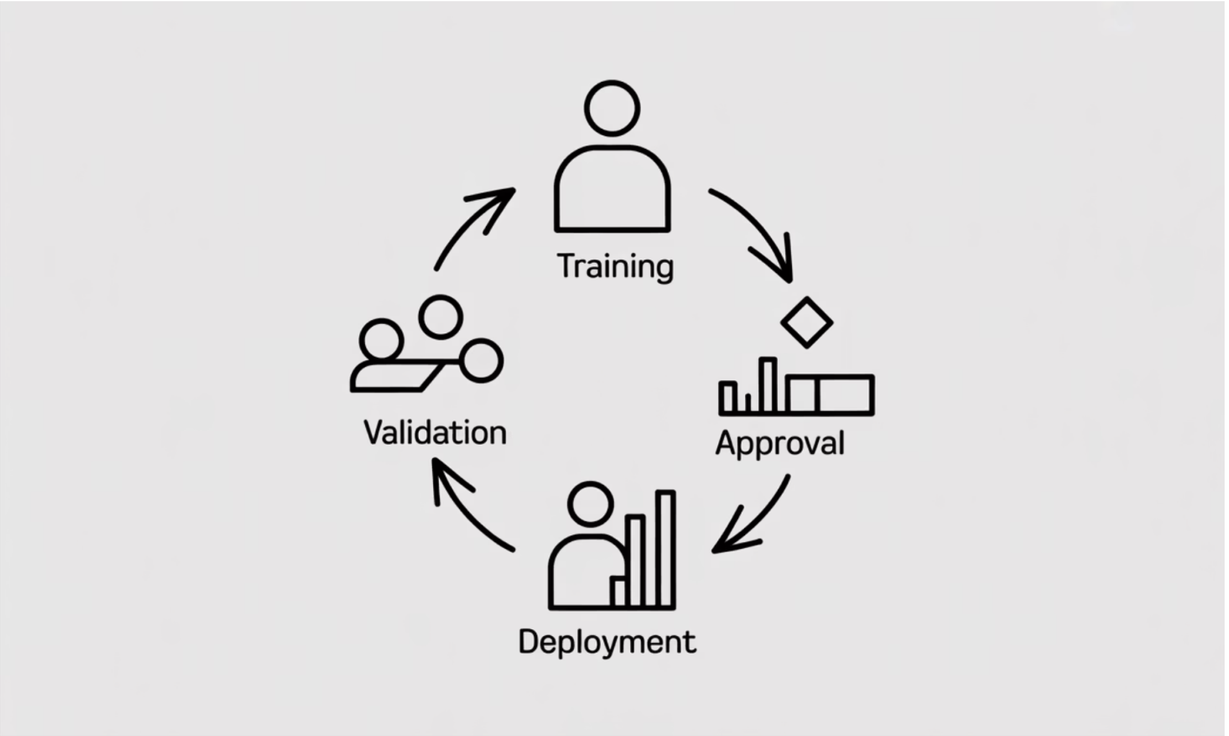
💡 В отличие от DAG-подхода (Airflow), BPMN — **декларативный**, ориентированный на **бизнес-семантику**, а не только на код.

BPMN управляет жизненным циклом модели в SAS Viya

SAS Workflow Manager — BPMN-движок внутри SAS Viya

Процесс:

1. Запуск retraining по триггеру
2. Валидация модели аналитиком
3. Согласование с бизнесом
4. Деплой в продакшн
5. Мониторинг → возврат к шагу 1 при дрифте



Основные объекты BPMN

Service Task: автоматизация шагов пайплайна

Выполняет **скрипты**, **API-вызовы**, **уведомления**

Примеры в ModelOps:

Python-скрипт

Запуск Python-скрипта обучения

Метрики

Отправка метрик в Neptune.ai

Деплой

Вызов REST API для деплоя модели

Технически:

- → В SAS: вызов SAS Job / REST endpoint
- → В Airflow: аналог — PythonOperator / HttpOperator

User Task: когда человек — часть процесса

Требует **ввода от пользователя**:

- Подтверждение качества модели
- Согласование бизнес-метрик
- Аудит решений

В SAS Workflow Manager:

→ Генерирует форму с полями, кнопками «Одобрить» / «Отклонить»

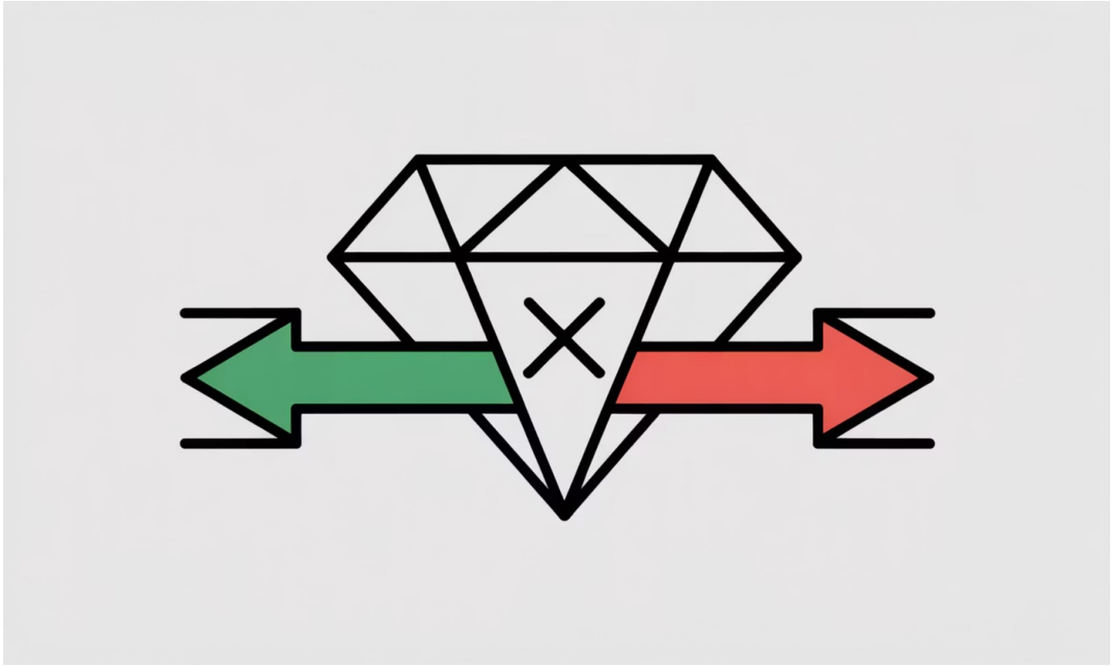
→ Назначается по ролям (аналитик, risk-officer, product-owner)



Шлюзы (Gateways) — логика принятия решений

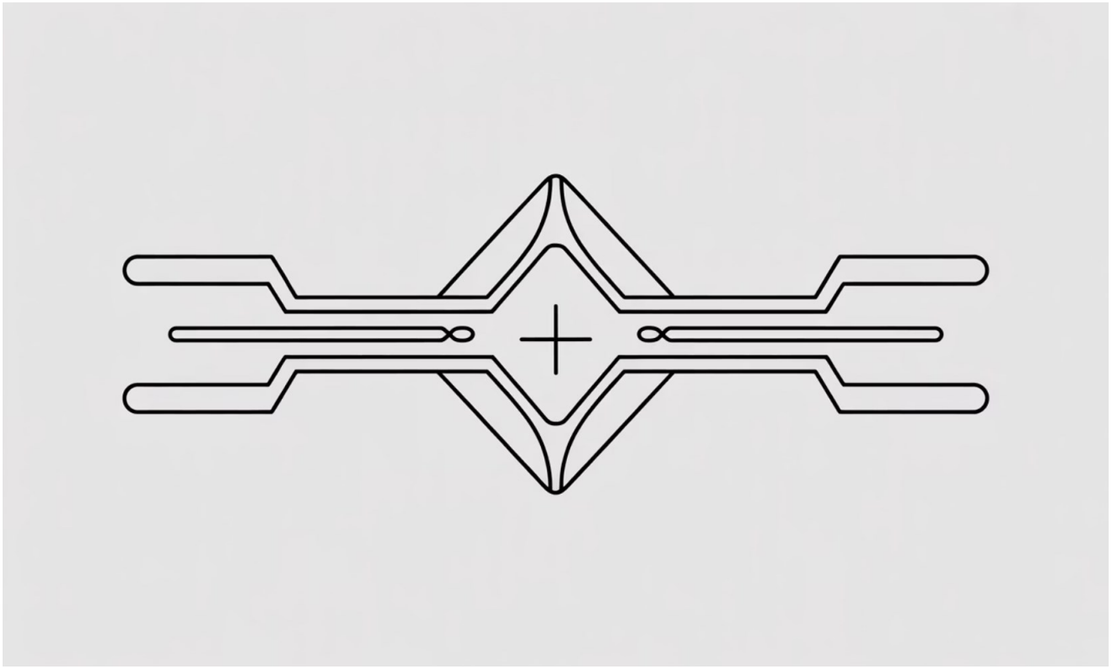
Exclusive Gateway — выбор одной ветки (if/else)

Пример: если AUC > 0.85 → деплой, иначе → доработка



Parallel Gateway — запуск нескольких веток одновременно

Пример: одновременная валидация и документирование



В BPMN: → Обозначается ромбом с символом «X» (exclusive) или «+» (parallel)

Подпроцессы — модульность процессов

Группировка шагов в **логический блок**

Примеры:

1

Валидация модели

расчет метрик + сравнение с baseline + генерация отчета

2

Деплой

тестирование API + blue/green rollout

Преимущества:

- Повторное использование
- Упрощение основной схемы
- Локальная ответственность

Как BPMN-объекты выглядят в SAS

Объект BPMN	Реализация в SAS
Service Task	вызов SAS Job или REST
User Task	форма с полями и ролями
Gateway	условие на метрику (например, model_drift > 0.1)
Subprocess	вложенный workflow

Совет: → Используйте переменные процесса (process variables) для передачи данных между шагами (например, model_id, auc_score)

Циклическая природа процессов ModelOps

От развертывания к циклу улучшения

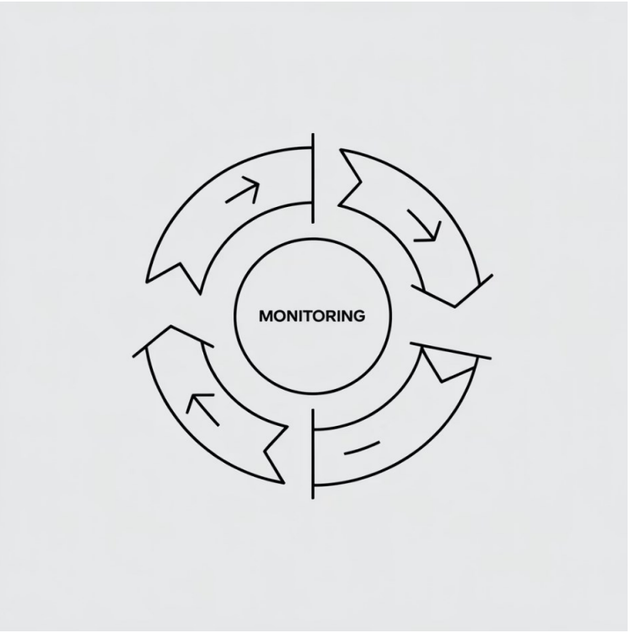
❌ Старый подход: «обучил → задеплоил → забыл»

✅ ModelOps:

Мониторинг → триггер → retraining → валидация → деплой → мониторинг...

BPMN позволяет:

→ Замкнуть процесс в **циклическую схему** с возвратом на шаг retraining



Когда запускать переобучение?

Data Drift

изменение распределения входных данных

Concept Drift

изменение связи между X и y

Accuracy Decay

падение метрик на продакшн-данных

Статистические тесты:

- Колмогоров-Смирнов (KS)
- PSI (Population Stability Index)

Пороги задаются в BPMN как условия в Gateway

Пример условия: IF PSI > 0.25 THEN retrain

Оркестрация retraining в BPMN

01

Start Event

по расписанию или триггеру

02

Service Task

загрузка новых данных

03

Service Task

обучение модели

04

User Task

валидация

05

Gateway

AUC > 0.8?

06

Да → деплой

Нет → уведомление + логирование

07

End

но с возвратом к мониторингу!

Retraining-цикл в Airflow

Хотя Airflow не использует BPMN напрямую, DAG можно спроектировать по BPMN-принципам:

Как эмулируется BPMN:

- BranchPythonOperator = **Exclusive Gateway**
- SlackAPIPostOperator = **User Task (с внешним согласованием)**
- Повторный запуск DAG = **цикл** (можно триггерить через ExternalTaskSensor или REST API)

```python
with DAG('model_retraining_cycle', schedule_interval=None) as dag:
    check_drift = PythonOperator(task_id='detect_drift', python_callable=check_psi)

    retrain = PythonOperator(task_id='retrain_model', python_callable=train_model)

    validate = SlackAPIPostOperator(
        task_id='request_human_validation',
        channel='#ml-team',
        text='Новая модель готова к валидации. AUC: {{ ti.xcom_pull("evaluate") }}'
    )

    evaluate = PythonOperator(task_id='evaluate_model', python_callable=calc_auc)

    decide = BranchPythonOperator(
        task_id='approve_or_reject',
        python_callable=lambda **ctx: 'deploy_model' if ctx['ti'].xcom_pull('evaluate') > 0.85 else 'notify_rejection'
    )

    deploy = PythonOperator(task_id='deploy_model', python_callable=deploy_to_prod)
    notify_rejection = PythonOperator(task_id='notify_rejection', python_callable=send_alert)

    # Логика цикла
    check_drift >> retrain >> evaluate >> decide
    decide >> [deploy, notify_rejection]
```

Как не утонуть в циклах?

⚠ Опасность:

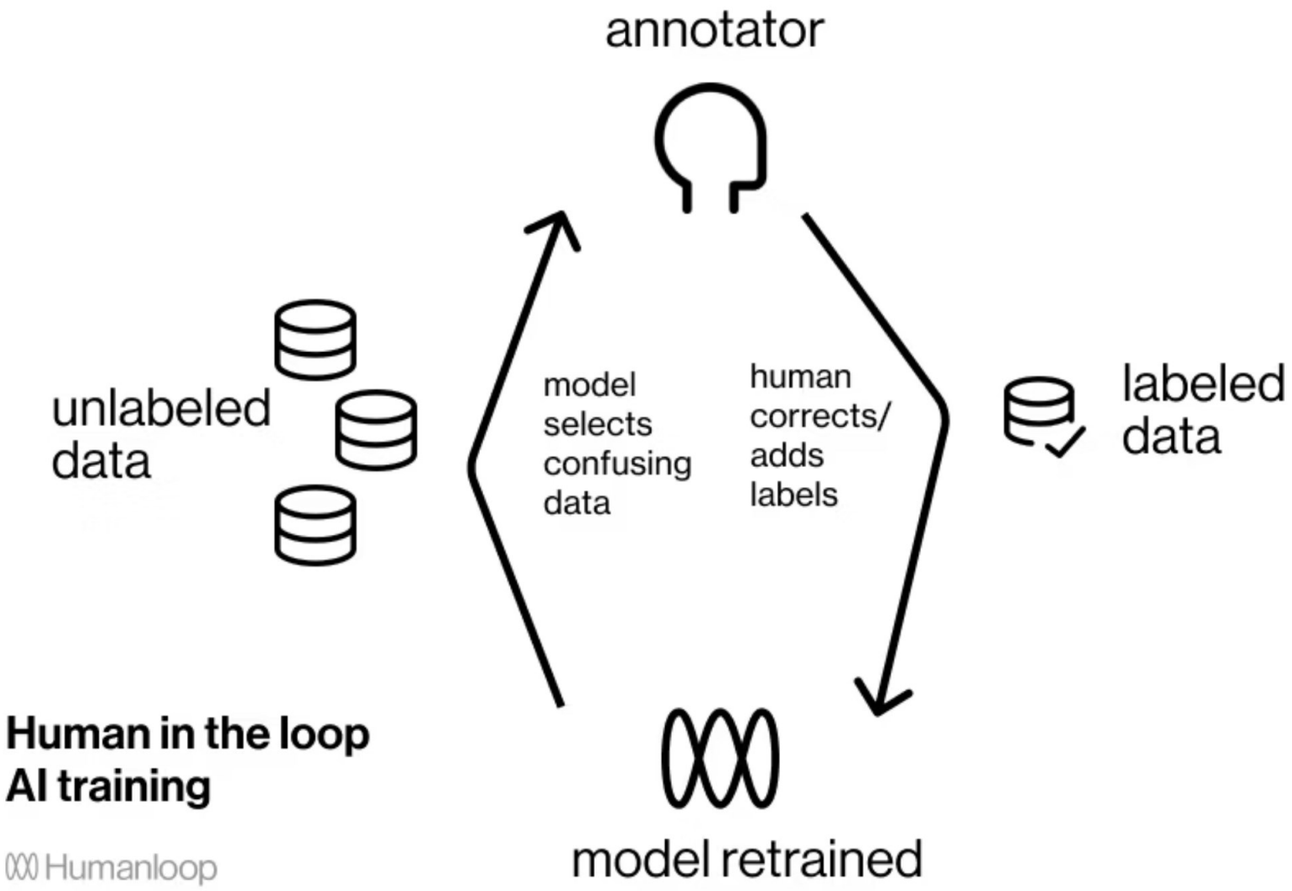
бесконечный retraining без улучшения

✅ Решение:

- Ограничьте количество попыток (max 3 retraining подряд)
- Введите **калибровочный этап**: если метрики не растут — приостановить цикл
- Добавьте **человеческое решение**: «Требуется анализ причины дрифта»

> «Цикл должен улучшать, а не крутить воду»

Human-in-the-loop в ModelOps



Роль человека в ModelOps

Где нужен human-in-the-loop:

- Валидация качества модели (особенно в high-stakes: финансы, медицина)
- Согласование бизнес-метрик
- Аудит решений (GDPR, explainability)
- Принятие решений при неопределенности

BPMN делает это управляемым и отслеживаемым



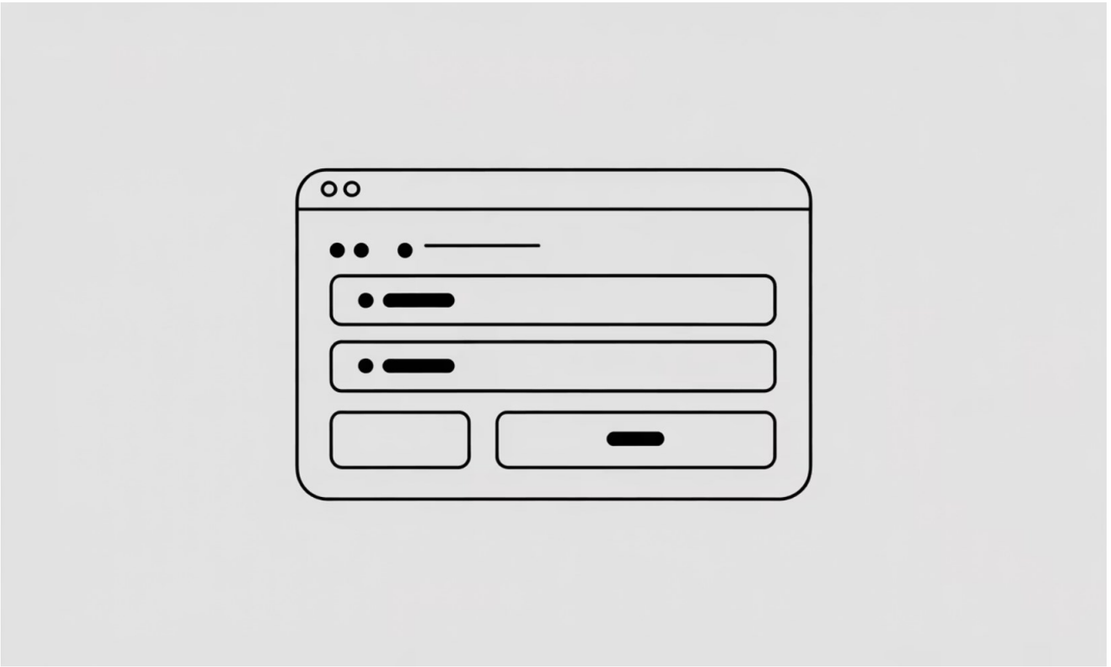
User Tasks: формы и роли

Форма содержит:

- Описание модели
- Метрики (AUC, F1, drift)
- Кнопки: «Одобрить», «Отклонить с комментарием»

Назначение по ролям:

- model_validator
- business_owner
- compliance_officer



Форма генерируется автоматически на основе BPMN

Уведомления и маршрутизация

При создании User Task → отправляется уведомление

Каналы

email, MS Teams, Telegram

Реализация

Service Task → вызов webhook

Пример:

→ «Новая модель ожидает валидации. Срок: 24 ч.»

→ Если не ответили — эскалация менеджеру

Интеграция с мессенджерами

- Создайте **вебхук** в Teams/Telegram
- В BPMN: Service Task → POST на webhook с JSON:
- Ответ → возвращается в BPMN как переменная

```json
{"text": "Модель v3.2 требует валидации", "buttons": ["✅ Одобрить", "❌ Отклонить"]}
```

Преимущество:Не нужно заходить в систему — решение в чате!

BPMN-процесс - от триггера до согласования



Оркестрация на практике

Единая точка управления жизненным циклом

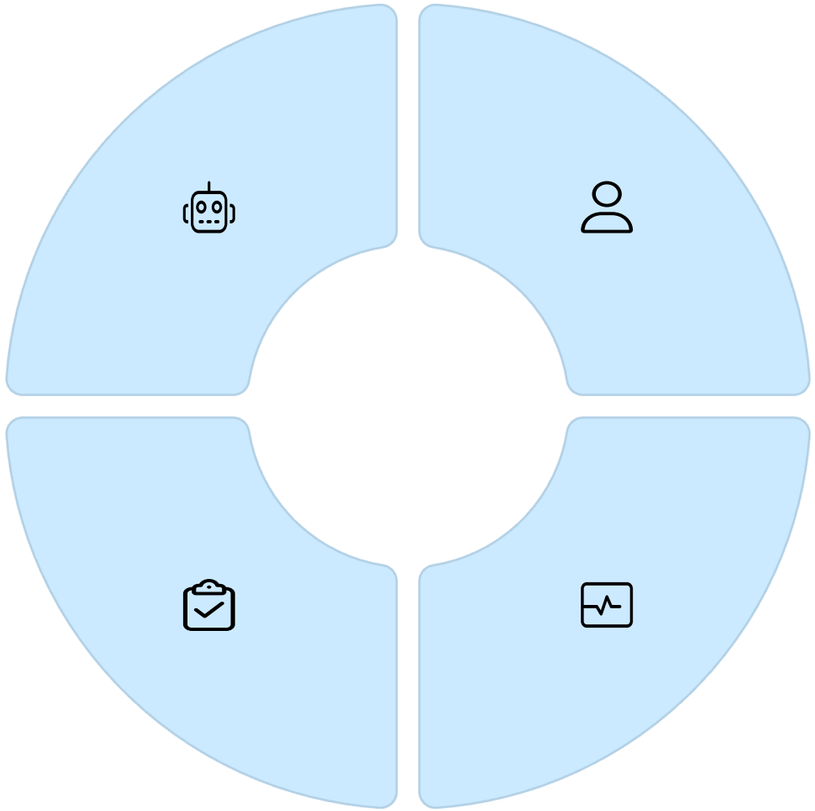
BPMN = **визуальный интерфейс** для оркестрации

Автоматизация

скрипты, API

Люди

формы, уведомления



Аудит

логи, версии

Мониторинг

триггеры

Результат: прозрачность, контроль, соответствие регуляторным требованиям

Open-source инструменты для BPMN-ориентированной оркестрации

Хотя Apache Airflow не поддерживает BPMN «из коробки», существуют **настоящие open-source BPMN-движки**, которые можно интегрировать в ModelOps:

🔷 Camunda Platform (Community Edition)

- Полная поддержка BPMN 2.0
- REST API для запуска процессов и передачи переменных
- Встроенный **Tasklist** для human-in-the-loop
- Можно запускать в Docker / Kubernetes

🔷 Flowable

- Лёгкий Java-движок с BPMN, CMMN, DMN
- Подходит для встраивания в микросервисы
- Поддержка асинхронных задач и вебхуков

🔷 Zeebe (от Camunda)

- Масштабируемый workflow-движок для high-load сценариев
- Использует **BPMN-like DSL**, но не полный BPMN 2.0
- Хорошо работает с Kafka и Kubernetes

🔷 Интеграция с ML-стеком:

- Вызов Python-скриптов через REST или сообщения (RabbitMQ/Kafka)
- Передача model_id, drift_score, auc как переменных процесса
- Уведомления через вебхуки в Telegram/Slack

> 💡 **Практический подход**: Используйте **Camunda Modeler** для проектирования → разверните **Camunda Engine** → интегрируйте с MLflow/Neptune через API.

Airflow: BPMN-инициированная оркестрация

- Airflow использует **DAGs**, а не BPMN
- Но можно эмулировать BPMN-логику:
- BranchPythonOperator = Gateway
- PythonOperator = Service Task
- ExternalTaskSensor + SlackOperator = User Task (частично)

Минус: нет встроенной поддержки форм и ролей

BPMN-элемент	Airflow-оператор
Service Task	PythonOperator
Gateway	BranchPythonOperator
User Task	SlackOperator

Сравнение: Enterprise vs Open-source

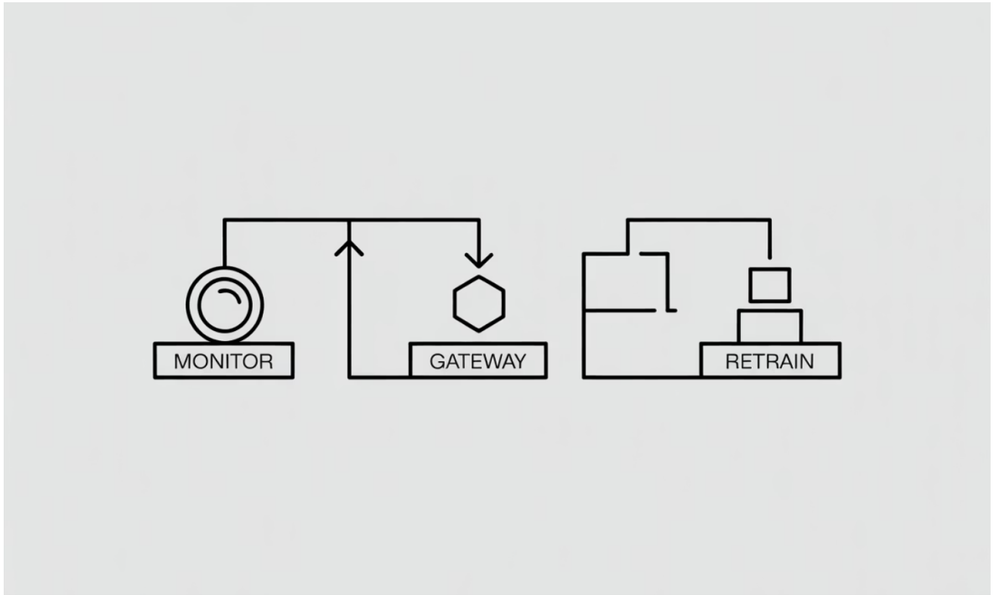
Критерий	SAS Viya (Enterprise)	Airflow (Open-source)
BPMN-поддержка	✅ Полная	❌ Только эмуляция
Human-in-the-loop	✅ Встроенные формы	⚠️ Через Slack/Email
Аудит и compliance	✅ Готово	❌ Требует доработки
Сложность внедрения	Высокая (стоимость)	Низкая (но больше ручной работы)

Демо: DAG как BPMN-процесс

Пример DAG для retraining-цикла

python

```python
retrain = PythonOperator(task_id='retrain_model', ...)
validate = SlackAPIPostOperator(
    task_id='request_validation',
    message='Модель готова к валидации'
)
deploy = BranchPythonOperator(
    task_id='check_quality',
    python_callable=lambda: 'deploy' if auc > 0.85 else 'notify_fail'
)
```



Вывод: → Возможна эмуляция, но без визуальной семантики BPMN

Управление и мониторинг

KPI для BPMN в ModelOps

24h	<24h	15%	3x
Время выполнения	**SLA**	**Отклонено**	**Частота**
процесса	по согласованию	моделей	триггеров в день

Инструменты:

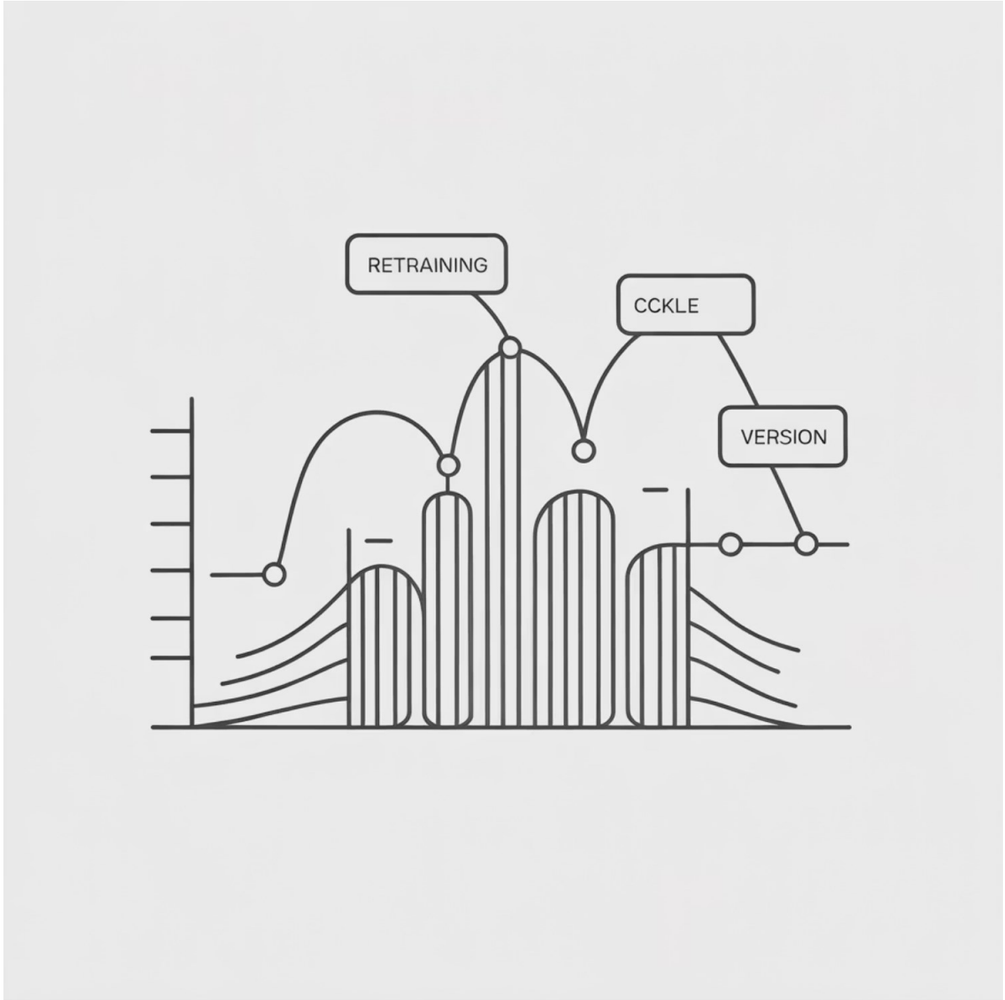
- SAS Environment Manager
- Prometheus + Grafana (для Airflow)
- Встроенные дашборды BPMN-движков

Связка BPMN + Neptune.ai для полного контроля

- BPMN запускает эксперимент → логирует в Neptune
- Neptune возвращает метрики → BPMN принимает решение

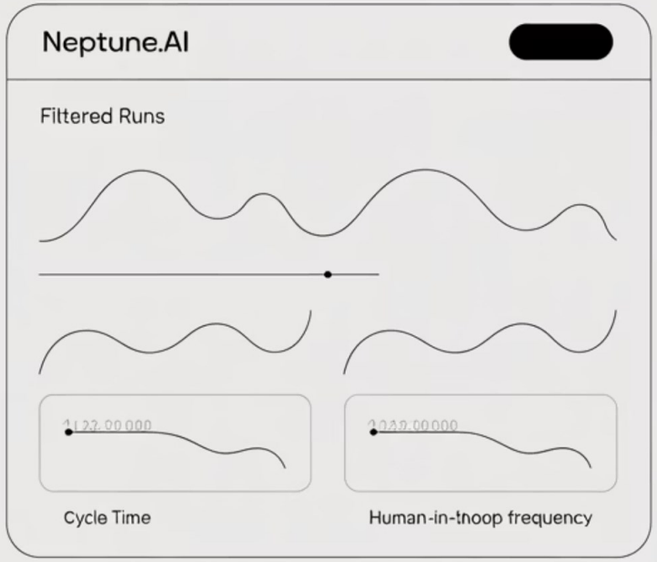
Визуализация:

- История метрик моделей
- Сравнение версий
- Связь с этапами BPMN-процесса



Как выглядит оркестрация в Neptune?

- Каждый запуск BPMN → новый run в Neptune
- Теги: `bpmn_process_id`, `model_version`, `trigger_type`
- Custom charts: время цикла, частота human-in-the-loop



BPMN превращает ModelOps в управляемую систему

Масштабируемость

повторное использование подпроцессов

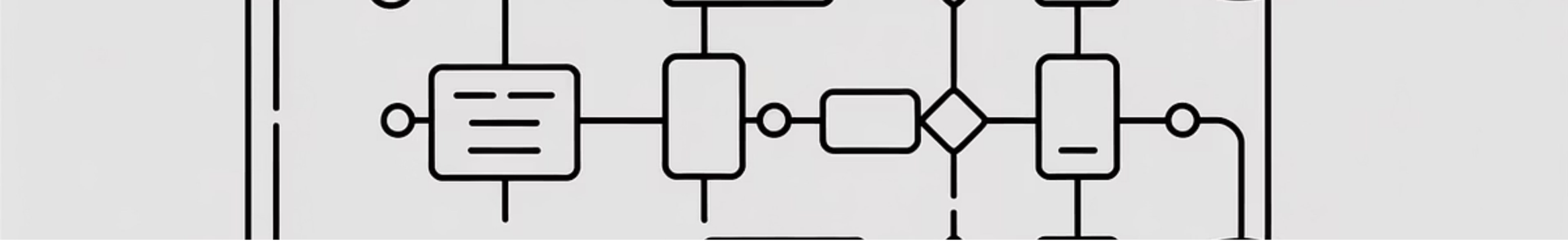
Контроль

каждый шаг — видим, измерим, аудируем

Прозрачность

единый язык для бизнеса и инженеров

> *BPMN — это **исполняемая спецификация** жизненного цикла модели.*



Домашнее задание

Спроектируйте BPMN-схему retraining-цикла

Задание:

1. Создайте BPMN-процесс, включающий:
   - Триггер по дрифту
   - Retraining
   - Валидацию (User Task)
   - Сравнение двух моделей (аукцион по AUC)
   - Деплой лучшей
   - Уведомление в Telegram
2. Используйте: https://bpmn.io/

Критерии: полнота, логика, human-in-the-loop